Data Structure Reference Notes

Certainly. To ensure you have enough material for a **15-mark answer**, Unit 1 is broken down into the fundamental definitions of Data Structures, their classifications, and the detailed logic for Stacks.

Unit 1: Introduction to Data Structures

1. Data Structure: Definition and Basic Concepts

A **Data Structure** is a systematic way of organizing, managing, and storing data in a computer so that it can be accessed and modified efficiently. It is not just about storing data but also defines the relationship between data elements and the operations allowed on them.

Basic Terminology:

- **Data:** A single value or a set of values (e.g., a student's marks).
- **Data Item:** A single unit of values.
- **Group Items:** Data items that can be divided into sub-items (e.g., Name divided into First Name and Last Name).
- **Elementary Items:** Data items that cannot be divided (e.g., Roll Number).
- **Entity and Attribute:** An entity is an object (e.g., Employee) and attributes are its properties (e.g., Salary, ID).

2. Importance of Data Structures in Programming

For large-scale applications, data structures are essential because:

1. **Data Search:** Efficiently searching through millions of records.
2. **Processor Speed:** Handling multiple requests simultaneously by organizing data logic.
3. **Multiple Requests:** Managing data when thousands of users access a database at once.

4. **Reusability:** Once a data structure is implemented, it can be used across different programs.

3. Classification of Data Structures

Data structures are broadly divided into **Primitive** and **Non-Primitive**.

A. Primitive Data Structures

These are the basic data types predefined by the programming language. They operate directly on machine-level instructions.

- **Integer:** Stores whole numbers.
- **Float:** Stores fractional/decimal numbers.
- **Character:** Stores single symbols or letters.
- **Pointer:** Stores the memory address of another variable.

B. Non-Primitive Data Structures

These are complex data structures derived from primitive ones.

1. **Linear Data Structures:** Elements are arranged in a sequential manner.
    - **Arrays:** Fixed-size, contiguous memory locations.
    - **Linked Lists:** Dynamic size, elements connected by pointers.
    - **Stacks:** Follows LIFO (Last In First Out).
    - **Queues:** Follows FIFO (First In First Out).
2. **Non-Linear Data Structures:** Elements are arranged in a hierarchical or interconnected way.
    - **Trees:** Parent-child relationship.
    - **Graphs:** Network of nodes and edges.

The Stack

1. Definition and Concept

A **Stack** is an abstract data type and a linear data structure that follows the **LIFO (Last In First Out)** principle. This means the last element added to the stack will be the first one to be removed. All operations (insertion and deletion) are performed at one end called the **TOP**.

2. Memory Representation of Stack

In C++, a stack can be represented using:

- **Array Representation:** A static implementation where a fixed-size array is used. A variable TOP is used to keep track of the index of the last element.
- **Linked List Representation:** A dynamic implementation where each element is a node. TOP points to the head of the list.

3. Algorithms for Stack Operations

For a 15-mark answer, you must provide the logic for **PUSH** and **POP**.

A. PUSH Algorithm (Insertion)

This operation adds an element to the top of the stack.

1. **Check for Overflow:** If TOP == MAX - 1, the stack is full. Exit with an error.
2. **Increment TOP:** TOP = TOP + 1.
3. **Insert Element:** Stack[TOP] = item.
4. **End.**

B. POP Algorithm (Deletion)

This operation removes the topmost element.

1. **Check for Underflow:** If TOP == -1, the stack is empty. Exit with an error.
2. **Access Element:** item = Stack[TOP].
3. **Decrement TOP:** TOP = TOP - 1.
4. **Return item.**
5. **End.**

4. Applications of Stack

To score high, detail how stacks are used in real computing:

- **Expression Conversion:** Converting Infix (A+B) to Postfix (AB+) or Prefix (+AB).
- **Expression Evaluation:** Computing the result of postfix expressions.
- **Function Calls:** The system uses a "Call Stack" to remember where to return after a function finishes.
- **Recursion:** Storing the state of each recursive call.
- **Undo Mechanism:** Used in text editors (like MS Word) to reverse the last action.
- **Backtracking:** Used in puzzle solving (like the N-Queens problem or Maze exploration).

Unit 2: Queues and Linked Lists

1. The Queue: Definition and Representation

A **Queue** is a linear data structure that follows the **FIFO (First-In-First-Out)** principle. The first element added to the queue will be the first one to be removed.

- **Front:** The end from which elements are deleted.
- **Rear:** The end where elements are inserted.

Memory Representation:

- **Array:** Uses a fixed-size array where Front and Rear are indices.
- **Linked List:** Uses nodes where Front points to the head and Rear points to the tail.

2. Types of Queues

- **Linear Queue:** Standard queue; once Rear reaches the end, we cannot insert even if space is available at the front (False Overflow).
- **Circular Queue:** The last position is connected to the first position.

    - *Condition for Full:* (Rear + 1) % MAX == Front.
- **Priority Queue:** Elements are processed based on priority.
- **Double Ended Queue (Deque):** Insertion/Deletion possible at both ends.

3. Queue Algorithms

Enqueue (Insertion)

1. Check for Overflow: If (Rear + 1) % MAX == Front, print "Queue Full".
2. If Front == -1, set Front = 0, Rear = 0.
3. Else, Rear = (Rear + 1) % MAX.
4. Queue[Rear] = element.

Dequeue (Deletion)

1. Check for Underflow: If Front == -1, print "Queue Empty".
2. element = Queue[Front].
3. If Front == Rear, set Front = -1, Rear = -1 (Queue becomes empty).
4. Else, Front = (Front + 1) % MAX.

4. Linked Lists: Definition and Types

A **Linked List** is a linear, dynamic data structure where elements (nodes) are not stored in contiguous memory locations. Each node has a **Data** part and a **Link (Next)** part.

Types:

1. **Singly Linked List (SLL):** Navigation is only forward. Last node points to NULL.
2. **Doubly Linked List (DLL):** Each node has two pointers (Next and Prev). Allows two-way traversal.
3. **Circular Linked List:** The last node points back to the first node.

Unit 3: Trees and Advanced Structures

1. Binary Tree: Definition and Traversal

A **Binary Tree** is a hierarchical data structure where each node has at most two children, referred to as the left child and the right child.

Traversal Algorithms (The "Must-Know"):

- **In-Order (Left, Root, Right):** Gives nodes in ascending order in a BST.
- **Pre-Order (Root, Left, Right):** Used to create a copy of the tree.
- **Post-Order (Left, Right, Root):** Used to delete the tree or evaluate postfix expressions.

2. Binary Search Tree (BST)

A BST is a binary tree where:

- The Left Subtree contains values smaller than the Root.
- The Right Subtree contains values larger than the Root.

Operations:

- **Insertion:** Compare value with root, move left or right recursively until an empty spot is found.
- **Deletion:** 1. *Leaf Node:* Delete directly.

2. *One Child:* Link child to grandparent.

3. *Two Children:* Replace with In-order Successor (smallest in right subtree) or Predecessor.

3. Advanced Tree Structures

AVL Trees (Balanced BST)

Standard BSTs can become skewed (like a linked list). AVL trees prevent this by maintaining a **Balance Factor** (Height of Left Subtree - Height of Right Subtree).

- **Balance Factor** must be **-1, 0, or 1**.
- Balanced using **Rotations** (LL, RR, LR, RL).

B-Trees

A multi-way search tree where a node can have more than two children.

- **Application:** Used heavily in **Databases and File Systems** because they minimize disk reads for large amounts of data.

Unit 4: Graphs, Sorting, and Searching

1. Graphs: Definition and Representation

A **Graph** $G = (V, E)$ consists of Vertices (V) and Edges (E).

- **Adjacency Matrix:** A 2D array where A[i][j] = 1 if there is an edge. Space: $O(V^2)$.
- **Adjacency List:** An array of linked lists. Space: $O(V + E)$.

Graph Traversals:

1. **Breadth-First Search (BFS):** Uses a **Queue**. Level-by-level exploration.
2. **Depth-First Search (DFS):** Uses a **Stack** (or Recursion). Explores as deep as possible before backtracking.

2. Sorting and Searching (Algorithms)

Technique	Logic	Complexity
Linear Search	Scan each element one by one.	$O(n)$
Binary Search	Divide and conquer (Sorted array only).	$O(\log n)$
Bubble Sort	Swap adjacent elements if they are in wrong order.	$O(n^2)$
Selection Sort	Find minimum and put it at the beginning.	$O(n^2)$

3. Heap Data Structure

A **Heap** is a complete binary tree.

- **Max Heap:** Root is the largest element.
- **Min Heap:** Root is the smallest element.
- **Applications:** Used in **Heap Sort** and to implement **Priority Queues**.